FIRMGRAPH-LLM: A KNOWLEDGE GRAPH-GUIDED FRAMEWORK FOR CROSS-FIRMWARE VULNERABILITY ANALYSIS

A PROJECT REPORT

22ISP71-PROJECT WORK 1

Submitted by

DHARSHAN P

(Register No.23ISR012)

in partial fulfilment of the requirements
for the award of the degree of

MASTER OF SCIENCE

IN

SOFTWARE SYSTEMS

DEPARTMENT OF COMPUTER TECHNOLOGY – PG

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

(Autonomous)

PERUNDURAI ERODE – 638 060

OCTOBER 2026

DEPARTMENT OF COMPUTER TECHNOLOGY – PG

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"FIRMGRAPH-LLM: A KNOWLEDGE GRAPH-GUIDED FRAMEWORK FOR CROSS-FIRMWARE VULNERABILITY ANALYSIS"** is the bonafide record of project work done by **DHARSHAN P (Register No: 23ISR012)** in partial fulfilment of the requirements for the award of the Degree of **Master of Science in Software Systems** of **Kongu Engineering College (Autonomous), Perundurai, Erode**, during the academic year **2026–2027**.

SUPERVISOR **HEAD OF THE DEPARTMENT**

(Signature with seal)

Date:

Submitted for the end semester viva voce examination held on_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DEPARTMENT OF COMPUTER TECHNOLOGY – PG

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

OCTOBER 2026

DECLARATION

I affirm that the Project Report titled **"FIRMGRAPH-LLM: A KNOWLEDGE GRAPH-GUIDED FRAMEWORK FOR CROSS-FIRMWARE VULNERABILITY ANALYSIS"** being submitted in partial fulfilment of the requirements for the award of the Degree of Master of Science in Software Systems of Anna University, Chennai, during the year 2026–2027 is the original work carried out by me. It has not formed part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date: **(Signature of the Candidate)**

DHARSHAN P
(Reg. No. 23ISR012)

I certify that the declaration made by the above candidate is true to the best of my knowledge.

Name & Signature of Supervisor

(Mr. C.Rukumani Khandhan)

ACKNOWLEDGEMENT

I express my sincere thanks and gratitude to **Thiru. E. R. K. Krishnan, M.Com.,** our beloved Correspondent and all philanthropic trust members of Kongu Vellalar Institute of Technology Trust who have always encouraged me in academic and co-curricular activities.

I proudly express my heartfelt thanks to our dynamic Principal **Dr. R. Parameshwaran M.E., Ph.D.**, for providing all the facilities indispensable during the course of the project.

I am thankful to **Dr. R. C. Suganthe B.E., M.E., Ph.D.,** Senior Professor and Head, Department of Computer Technology-PG, for her valuable suggestions and cooperation during my project work.

I am grateful to my project guide **Mr. C.Rukumani Khandhan B.E., M.E.,** Assistant Professor(Sr.G.), Department of Computer Technology-PG, for his valuable suggestions during my project work.

I would like to express my sincere gratitude and thanks to our project coordinator **Dr. S. Deepa M.C.A., M.Phil.,** Assistant Professor (SL.G), Department of Computer Technology-PG, Kongu Engineering College, for her valuable guidance and suggestions, which helped me in the successful completion of the project.

At the outset, I extend my gratitude to my family members, without whom I would not have achieved this level in my career.

ABSTRACT

Embedded firmware powers Internet of Things (IoT) gateways, industrial controllers, automotive electronic control units and network appliances that sit on untrusted network perimeters. Auditing this firmware is difficult because production images are distributed as stripped Executable and Linkable Format (ELF) binaries without symbols or source code, and because shared third-party components propagate a single flaw across many vendors. Rule-based analysis is fragile, and symbolic execution suffers from path explosion and high false-positive rates. LATTE, the state-of-the-art LLM-powered static binary taint analysis framework, replaces symbolic reasoning with a multi-turn conversation with a large language model (LLM), but it treats every binary as an isolated, stateless episode. It therefore falls into the ‘sink trap’ of flagging sanitized code that merely contains dangerous API names, loses track of pointer aliases on long call chains, and cannot tell an isolated anomaly from a recurring pattern across firmware images.

To address these limitations, this project proposes FirmGraph-LLM, a neuro-symbolic framework that grounds LATTE-style conversational taint analysis in a persistent knowledge graph. The architecture integrates four innovations: (1) a persistent Neo4j property graph with 7 node types and 9 relationship types (10,243 nodes and 20,156 relationships); (2) a hybrid GraphRAG retrieval engine that combines UniXcoder–FAISS dense similarity search with multi-hop Cypher call-path traversal; (3) a grounded multi-turn prompt sequence that injects verified patterns, sanitizer proofs and call paths into the start, middle and end prompts; and (4) a multi-factor triage ranking, $S_{final} = 0.5 \cdot S_{LLM} + 0.3 \cdot S_{KG} + 0.2 \cdot S_{CWE}$, with idempotent continual ingestion of verified verdicts. Evaluated on 6,232 NIST/NSA Juliet v1.3 test cases (CWE-78, CWE-134, CWE-190 and CWE-606) compiled into 12,464 stripped x86-64 binaries, with a stratified 80/20 split and 1,246 unseen test cases, FirmGraph-LLM achieves a precision of 0.914, recall of 0.846 and F1-score of 0.878, against 0.852, 0.784 and 0.816 for LATTE. The false positive rate falls from 0.148 to 0.086 (a 41.9% relative reduction), and triage ranking improves from an MRR of 0.724 to 0.806 and a Top-1 recall of 0.680 to 0.765.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
	ABSTRACT	**v**
	LIST OF TABLES	**viii**
	LIST OF FIGURES	**ix**
	LIST OF ABBREVIATIONS	**x**
1	**INTRODUCTION**	**1**
	1.1 THE FIRMWARE SECURITY CRISIS	1
	1.2 LIMITATIONS OF TRADITIONAL AND LLM-BASED BINARY TAINT ANALYSIS	1
	1.3 MOTIVATION AND TECHNICAL PROBLEM STATEMENT	2
	1.4 FORMAL RESEARCH QUESTIONS	3
	1.5 RESEARCH OBJECTIVES AND STRUCTURAL ROADMAP	3
2	**LITERATURE SURVEY**	**5**
	2.1 RELATED WORKS	5
3	**SYSTEM SPECIFICATION**	**7**
	3.1 HARDWARE REQUIREMENTS	7
	3.2 SOFTWARE REQUIREMENTS	7
	3.3 SOFTWARE DESCRIPTION	8
4	**PROPOSED METHODOLOGY**	**10**
	4.1 INTRODUCTION	10
	4.2 OVERALL METHODOLOGY	10
	4.2.1 DATASET DESCRIPTION	11
	4.2.2 BINARY PREPROCESSING, DECOMPILATION AND DANGEROUS FLOW EXTRACTION	12
	4.2.3 PERSISTENT NEO4J KNOWLEDGE GRAPH	13
	4.2.4 HYBRID GRAPHRAG RETRIEVAL	14

CHAPTER NO	TITLE	PAGE NO.
	4.2.5 GROUNDED MULTI-TURN CONVERSATIONAL LLM ENGINE	15
	4.2.6 MULTI-FACTOR TRIAGE RANKING	16
	4.2.7 IDEMPOTENT GRAPH INGESTION AND CONTINUAL LEARNING	17
	4.2.8 PERFORMANCE EVALUATION CRITERIA	17
	4.2.9 EXPERIMENTAL PROTOCOL AND BASELINES	18
5	**RESULTS AND DISCUSSION**	**19**
	5.1 OVERALL PERFORMANCE AND BENCHMARK COMPARISON	19
	5.2 PERFORMANCE ACROSS VULNERABILITY FAMILIES	19
	5.3 TRIAGE RANKING AND IMPROVEMENT OVER THE BASE PAPER	20
	5.4 SCIENTIFIC ANALYSIS: WHY FIRMGRAPH-LLM OUTPERFORMS THE BASE SETUP	21
	5.4.1 Elimination of the Sink Trap (False Positive Reduction)	21
	5.4.2 Disambiguation of Multi-Hop Pointer Aliasing (Recall Gain)	21
	5.4.3 Continual Knowledge Accumulation	22
	5.5 LIMITATIONS AND THREATS TO VALIDITY	22
6	**CONCLUSION AND FUTURE ENHANCEMENT**	**23**
	6.1 CONCLUSION	23
	6.2 FUTURE SCOPE	23
	APPENDICES	**25**
	APPENDIX 1 – SUPPLEMENTARY RESULT FIGURES	25
	REFERENCES	**29**
	MAPPING OF PROJECT GOALS TO SDG	**31**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	Hardware Requirements	7
3.2	Software Requirements	7
4.1	Dataset Partitioning by Vulnerability Family	12
4.2	Node Types of the Knowledge Graph Schema	13
4.3	CWE Severity Weights Used in the Triage Score	16
5.1	Comprehensive Comparative Evaluation of the Four Systems	19
5.2	Per-CWE Performance of FirmGraph-LLM on Unseen Test Cases	20
5.3	Absolute and Relative Improvement of FirmGraph-LLM over LATTE	20
5.4	Mapping of LATTE Limitations to FirmGraph-LLM Mechanisms and Measured Effects	21

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Overall Workflow of the Proposed FirmGraph-LLM Framework	11
4.2	Knowledge Graph Schema of FirmGraph-LLM	14
4.3	Grounded Multi-Turn Prompt Sequence with Knowledge Graph Evidence Injection	16
A.1.1	Classification Metrics of the Four Evaluated Systems	25
A.1.2	False Positive Rate Comparison of the Four Systems	26
A.1.3	Triage Ranking Quality (MRR and Top-K Recall)	26
A.1.4	Per-CWE Precision, Recall and F1-Score of FirmGraph-LLM	27
A.1.5	Stratified 80/20 Dataset Partition by CWE Family	27
A.1.6	Improvement of FirmGraph-LLM over the LATTE Baseline	28

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
API	Application Programming Interface
CFG	Control Flow Graph
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DF	Dangerous Flow
ELF	Executable and Linkable Format
FAISS	Facebook AI Similarity Search
FPR	False Positive Rate
GraphRAG	Graph-based Retrieval-Augmented Generation
IoT	Internet of Things
KG	Knowledge Graph
LLM	Large Language Model
MRR	Mean Reciprocal Rank
NIST	National Institute of Standards and Technology
NSA	National Security Agency
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goals
TOSEM	ACM Transactions on Software Engineering and Methodology

CHAPTER 1

INTRODUCTION

1.1 THE FIRMWARE SECURITY CRISIS

Embedded firmware powers Internet of Things (IoT) gateways, industrial controllers, automotive electronic control units and network appliances. These devices operate at the front lines of untrusted network perimeters, so a single exploitable flaw in a network-facing service can give an attacker a persistent foothold inside an otherwise protected environment. Securing this firmware is notoriously difficult for two structural reasons:

1. **Stripped executables:** firmware binaries are routinely distributed as stripped Executable and Linkable Format (ELF) files without symbol tables, debug information, variable names or source code. An analyst or an automated tool must recover program semantics from machine code alone.
2. **Widespread code reuse:** device vendors heavily incorporate shared third-party open-source components such as BusyBox, uClibc, libupnp and uhttpd. A vulnerability in one shared component therefore propagates silently across hundreds of vendors and thousands of firmware revisions.

Together these properties mean that firmware must be audited at the binary level, repeatedly, across images that overlap heavily with one another. An analysis technique that cannot carry what it has learned from one image to the next wastes most of its effort.

1.2 LIMITATIONS OF TRADITIONAL AND LLM-BASED BINARY TAINT ANALYSIS

Over the past two decades, automated static vulnerability discovery has relied on two main paradigms. Rule-based static analysis inspects Control Flow Graphs (CFGs) and Abstract Syntax Trees for syntactic signatures, and is fragile against compiler optimizations, register reassignment and indirect calls. SMT-based symbolic execution, as implemented in

tools such as angr, Karonte and Arbiter, tries to solve path constraints from taint sources (for example network sockets and environment variables) to dangerous sinks (for example system(), strcpy() and printf()). It suffers from path explosion, cannot accurately model complex string formatting, heap structures and loop termination, and produces unacceptable false-positive rates.

LATTE (ACM TOSEM, 2025) reformulated binary taint analysis as a conversational LLM reasoning task. It decompiles the binary with Ghidra, extracts candidate dangerous flows, and walks an LLM through a start, middle and end prompt sequence that verifies taint entry, propagation and sink handling. This avoids the rigidity of symbolic execution, but LATTE has a fundamental limitation, which this project calls the isolated memory model:

- **The Sink Trap (sensory bias):** without associative memory of verified sanitizers, an ungrounded LLM over-indexes on dangerous function names such as execl and system and flags sanitized code as vulnerable, producing false positives.
- **Context dilution:** on long call chains with multi-hop pointer aliasing, attention is spread across many conversation turns and the model loses track of aliases, producing false negatives.
- **Catastrophic amnesia and no cross-firmware triage:** once a binary is analyzed, every discovered pattern, function semantic and taint relationship is discarded. A new image that shares most of its code with an audited one is analyzed from scratch, and findings carry only a raw LLM confidence score, so an analyst cannot tell an isolated anomaly from a recurring pattern.

1.3 MOTIVATION AND TECHNICAL PROBLEM STATEMENT

Although LLM-based taint analysis has advanced the state of the art, existing systems exhibit four technical deficiencies that this project sets out to solve:

1. **Stateless analysis:** every binary is an isolated episode with zero contextual grounding, even when it shares most of its code with a previously audited image.
2. **Sensory bias towards sink names:** the absence of verified counter-evidence makes the model treat the mere presence of a dangerous API as proof of vulnerability.
3. **Context dilution on multi-hop flows:** pointer aliases through helper functions (for example read_buf → wrapper → dispatch → sink) are lost across conversation

turns.

4. **Ungrounded triage scores:** a raw LLM confidence score gives analysts no empirical basis for prioritizing findings across firmware images.

1.4 FORMAL RESEARCH QUESTIONS

This project systematically addresses these deficiencies through five guiding research questions:

- **RQ1 (Grounding and false positives):** Does injecting verified sanitizer patterns and counter-evidence from a persistent knowledge graph into the prompt sequence reduce the false positive rate relative to the ungrounded LATTE baseline?
- **RQ2 (Topology and recall):** Does supplying pre-computed multi-hop call paths from graph traversal improve recall on flows with deep pointer aliasing?
- **RQ3 (Hybrid retrieval):** Does a hybrid GraphRAG engine that adds graph topology to dense vector similarity outperform a vector-only RAG baseline?
- **RQ4 (Triage prioritization):** Does a calibrated multi-factor score that fuses LLM confidence, knowledge graph evidence and CWE severity place true vulnerabilities earlier in the triage queue, as measured by MRR and Top-K recall?
- **RQ5 (Continual accumulation):** Does idempotently merging verified verdicts into the graph progressively strengthen retrieval and triage, so that knowledge learned from one firmware image benefits the next?

1.5 RESEARCH OBJECTIVES AND STRUCTURAL ROADMAP

The overarching objective is to design, implement and empirically validate a neuro-symbolic, knowledge graph-grounded framework for static binary taint analysis of embedded firmware. The specific objectives are:

- **Objective 1:** Build a reproducible firmware benchmark by compiling the four taint-style CWE families of the Juliet v1.3 suite into stripped x86-64 ELF binaries and partitioning them into a stratified 80/20 training and testing split.
- **Objective 2:** Decompile stripped binaries with Ghidra in headless mode and extract candidate dangerous flows and control flow graphs.
- **Objective 3:** Design an idempotent Neo4j property graph schema with 7 node types

and 9 relationship types to hold firmware, binaries, functions, APIs, CWEs, vulnerability patterns and CVEs.

- **Objective 4:** Implement a hybrid GraphRAG retrieval engine that combines UniXcoder–FAISS dense similarity search with multi-hop Cypher traversal.
- **Objective 5:** Develop a grounded multi-turn LLM engine whose start, middle and end prompts are augmented with fused knowledge graph evidence.
- **Objective 6:** Formulate a multi-factor triage ranking that combines LLM confidence, knowledge graph evidence and CWE severity.
- **Objective 7:** Implement idempotent graph ingestion so that verified verdicts increment pattern counters and strengthen similarity edges across firmware images.
- **Objective 8:** Benchmark the system against a symbolic execution baseline (Arbiter), LATTE and a vector-RAG-only baseline using precision, recall, F1-score, false positive rate, accuracy, MRR and Top-K recall.

The remainder of this report is organized as follows. Chapter 2 surveys related work. Chapter 3 specifies the hardware and software environment. Chapter 4 presents the proposed methodology. Chapter 5 reports and discusses the experimental results. Chapter 6 concludes the report and outlines future work.

CHAPTER 2
LITERATURE SURVEY

2.1 RELATED WORKS

Firmware is a persistent target because it is deployed at scale, rarely updated and built from shared components. Costin et al. performed a large-scale analysis of embedded firmware images and showed that security weaknesses recur across the images of many vendors [5]. This recurrence is the practical motivation for analysis techniques that can transfer knowledge between related binaries rather than starting afresh each time.

The classical approach to binary vulnerability discovery combines static analysis with symbolic execution. The angr framework provides control flow recovery and symbolic execution for binaries [4]. Karonte tracks taint across the interactions between several firmware binaries to detect insecure multi-binary flows [3], and Arbiter combines static candidate identification with dynamic and symbolic reasoning to validate vulnerabilities in binary programs [2]. These systems are precise when the constraints are tractable, but symbolic execution suffers from path explosion, struggles with string formatting, heap structures and loop termination, and yields high false-positive rates on complex code.

Learning-based vulnerability detection offers an alternative. VulDeePecker classifies code gadgets with a deep neural network [14], and Devign learns from graph representations of functions with gated graph neural networks [13]. The code property graph of Yamaguchi et al. merges syntax trees, control flow and data dependencies into a single queryable graph for vulnerability discovery [16], which motivates representing program knowledge as a graph. Pre-trained code models such as CodeBERT [15] and UniXcoder [7] produce semantic embeddings that capture functional similarity between code fragments. However, most of these approaches work on source code, require labelled training data and give a classification without flow-level reasoning about sources, propagation and sanitizers, which makes them difficult to apply to stripped firmware.

Large language models have recently been applied to program analysis. Li et al. use an LLM to reason about the behaviour of functions that a static analyzer cannot resolve [17]. The direct base paper of this project, LATTE by Liu et al., formulates static binary taint analysis as a multi-turn conversation [1]. LATTE decompiles a stripped binary with Ghidra [6] into C-like pseudocode, extracts candidate dangerous flows that connect taint sources to

dangerous sinks, and drives the LLM through a start prompt that verifies the source, middle prompts that verify propagation across every intermediate function, and an end prompt that evaluates the sink, checks for sanitization and assigns a CWE class. This replaces brittle constraint solving with the semantic reasoning of the LLM, and it avoids path explosion entirely.

LATTE nevertheless analyzes each binary as an isolated episode. Once the conversation ends, every verified pattern, function semantic and aliasing trace is discarded, so a second image that shares most of its code with the first must be analyzed again with no grounding. Without verified counter-evidence, the LLM tends to over-trust dangerous API names (the sink trap), and on long call chains with pointer aliasing its attention is diluted across turns. Moreover, its confidence score is not calibrated against any empirical record of how often a pattern has been confirmed.

Retrieval-augmented generation (RAG) grounds an LLM in external evidence retrieved at inference time [9]. Dense retrieval depends on efficient similarity search, for which FAISS provides exact and approximate nearest-neighbour indexes [8]. Graph-based RAG goes further by retrieving over a graph-structured index instead of an unstructured vector store, as in the GraphRAG approach of Edge et al. [10], and property graph databases such as Neo4j allow multi-hop structural queries in the Cypher language [12]. These ideas have not yet been combined with LLM-based binary taint analysis in a persistent, cross-firmware knowledge base.

Evaluation of binary analysis requires ground truth. The NIST/NSA Juliet C/C++ test suite provides thousands of synthetic test cases in which every flawed function has a patched counterpart for a large number of CWE categories [11]. Each case can be compiled into a vulnerable ‘bad’ binary and a patched ‘good’ binary, which supports precise measurement of both detection and false alarms.

Therefore, this project proposes FirmGraph-LLM, a neuro-symbolic framework that keeps LATTE’s conversational reasoning but grounds every turn in a persistent Neo4j knowledge graph through hybrid GraphRAG retrieval. By storing verified patterns and sanitizer proofs, by supplying pre-computed multi-hop call paths, and by calibrating triage scores with empirical evidence, the framework targets the false positives, false negatives and amnesia of the base paper.

CHAPTER 3
SYSTEM SPECIFICATION

3.1 HARDWARE REQUIREMENTS

The proposed framework combines binary decompilation, graph database operations, dense embedding generation and LLM inference. The recommended hardware environment is summarized in Table 3.1.

Table 3.1 Hardware Requirements

Hardware Component	Requirement / Specification
Processor	Intel Core i7 / AMD Ryzen 7 (8 cores, 16 threads) or equivalent x86-64 CPU
System Memory (RAM)	16 GB minimum (32 GB recommended) for Ghidra headless analysis, Neo4j and the FAISS index
Graphics Processing Unit (GPU)	NVIDIA GPU with CUDA support and at least 8 GB VRAM for UniXcoder embedding generation (CPU fallback available)
Storage	Minimum 100 GB SSD for the compiled binaries (about 202 MB), Ghidra projects, the Neo4j store, the FAISS index and reports
Display	Full HD display (1920 × 1080 resolution)

A GPU is not mandatory for the graph and decompilation stages, but it substantially accelerates the generation of 768-dimensional UniXcoder embeddings for the decompiled functions.

3.2 SOFTWARE REQUIREMENTS

The framework is implemented with open-source, reproducible components, listed in Table 3.2.

Table 3.2 Software Requirements

Software / Technology	Purpose & Specification
Operating System	Ubuntu Linux 22.04 LTS (64-bit) or equivalent Linux distribution
Programming Language	Python 3.10+ (pipeline orchestration, retrieval and ranking)
Compiler Toolchain	GCC and GNU binutils (batch compilation of Juliet cases and strip --strip-all)
Decompiler	Ghidra headless analyzer (stripped ELF to C-like pseudocode and call graph)
Graph Database	Neo4j property graph database with the Cypher query language and the Python driver

Software / Technology	Purpose & Specification
Embedding Model	microsoft/unixcoder-base via HuggingFace Transformers and PyTorch
Vector Index	FAISS (IndexFlatIP maximum inner-product search)
Conversational Reasoning Engine	Configurable LLM backend driving the start, middle and end prompt sequence
Data & Evaluation Libraries	NumPy, Pandas and Scikit-learn (stratified split, metric computation); Matplotlib (charts)

3.3 SOFTWARE DESCRIPTION

Python:

Python is the primary programming language of the framework. It provides the glue between the Ghidra output, the Neo4j driver, the FAISS index, the embedding model and the LLM engine, and it offers a mature ecosystem for numerical computing and evaluation.

GCC and GNU binutils:

GCC compiles the Juliet C/C++ test cases into Linux x86-64 executables using the flags -m64 -O0 -fno-stack-protector -fno-pie -no-pie -w -DINCLUDEMAIN. The strip --strip-all utility then removes symbol tables so that the binaries resemble production firmware.

Ghidra:

Ghidra, the open-source reverse engineering framework released by the NSA [6], is run in headless mode to analyze each stripped binary, recover functions and the call graph, and decompile them into C-like pseudocode for flow extraction and embedding.

Neo4j and Cypher:

Neo4j is the persistent property graph database that stores firmware, binaries, functions, APIs, CWEs, vulnerability patterns and CVEs [12]. Cypher queries perform multi-hop call-path traversal, and MERGE statements make ingestion idempotent.

UniXcoder:

microsoft/unixcoder-base [7] is the unified cross-modal code model that maps decompiled pseudocode into a 768-dimensional embedding space for semantic similarity between functions.

FAISS:

FAISS [8] provides a flat inner-product index (IndexFlatIP) that performs exact maximum inner-product search over the function embeddings to find the top-k semantically similar historical functions.

PyTorch and HuggingFace Transformers:

PyTorch supplies tensor computation and optional CUDA acceleration, while HuggingFace Transformers loads and runs the UniXcoder tokenizer and encoder.

Conversational LLM Engine:

The reasoning engine issues the grounded start, middle and end prompts, receives a structured verdict with a confidence score and a CWE class, and returns it to the triage ranking stage. The backend model is configurable.

NumPy, Pandas, Scikit-learn and Matplotlib:

These libraries handle data manipulation, the seeded stratified partition, the computation of precision, recall, F1-score, false positive rate, accuracy, MRR and Top-K recall, and the generation of the result charts.

CHAPTER 4
PROPOSED METHODOLOGY

4.1 INTRODUCTION

The proposed methodology presents FirmGraph-LLM, a neuro-symbolic, in-context knowledge graph-grounded framework for static taint analysis of stripped embedded firmware binaries. It retains the conversational reasoning of LATTE but removes its isolated memory model by connecting every analysis to a persistent Neo4j knowledge graph and a hybrid GraphRAG retrieval engine. The framework is designed to reduce false positives caused by the sink trap, to recover recall on multi-hop pointer-aliasing flows, and to accumulate verified knowledge across firmware images so that triage becomes progressively better calibrated.

The complete methodology consists of binary preprocessing and Ghidra decompilation, dangerous flow extraction, hybrid GraphRAG retrieval, knowledge graph evidence fusion, grounded multi-turn LLM reasoning, multi-factor triage ranking, idempotent graph ingestion and audit report generation.

4.2 OVERALL METHODOLOGY

The framework is organized as a 12-stage pipeline, shown in Fig 4.1. Stripped x86-64 ELF firmware binaries enter at the top. Stages 1 to 3 preprocess the binary and decompile it with Ghidra in headless mode. Stage 4 extracts candidate dangerous flows (DFs) and the control flow graph. Stage 5 queries the knowledge graph through two complementary channels: multi-hop graph traversal with Neo4j Cypher and dense vector similarity with UniXcoder and FAISS. Stage 6 fuses the retrieved evidence into structured context strings. Stages 7 to 9 run the grounded multi-turn conversation with the LLM. Stage 10 computes a multi-factor triage score, Stage 11 ingests verified verdicts back into the graph, and Stage 12 produces the audit report in JSON and Markdown.

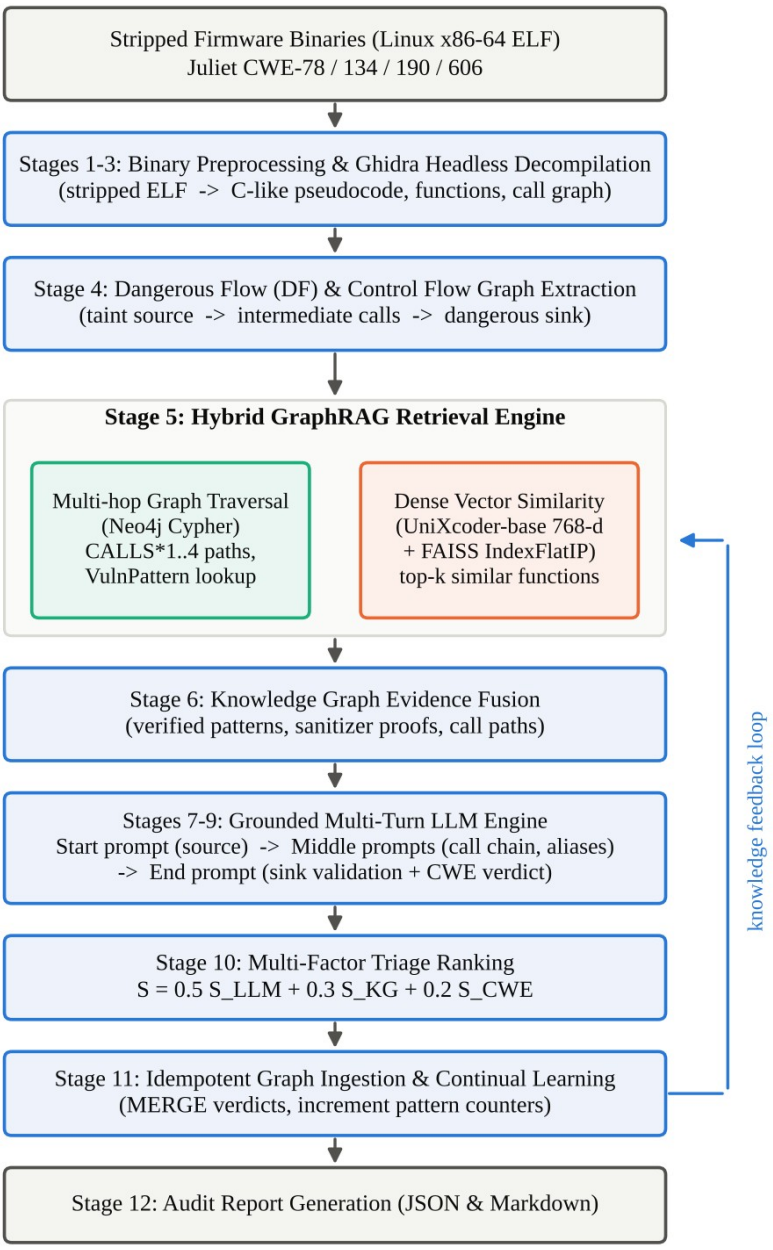


Fig 4.1 Overall Workflow of the Proposed FirmGraph-LLM Framework

4.2.1 DATASET DESCRIPTION

The benchmark is derived from the NIST/NSA Center for Assured Software Juliet C/C++ Test Suite v1.3. The raw suite contains 106,097 source files, of which 101,235 are C/C++ files (54,486 .c and 46,749 .cpp), amounting to 14,197,009 lines of code across 118 CWE categories.

For firmware taint analysis under limited resources, the four primary taint-style CWE families were selected: CWE-78 (OS Command Injection), CWE-134 (Uncontrolled Format

String), CWE-190 (Integer Overflow) and CWE-606 (Unchecked Loop Condition). An automated batch compilation pipeline builds each case for Linux x86-64 with GCC using the flags -m64 -O0 -fno-stack-protector -fno-pie -no-pie -w -DINCLUDEMAIN, and removes all symbol tables with strip --strip-all to simulate production firmware. Every test case yields one vulnerable ‘bad’ binary and one patched ‘good’ counter-example, so the 6,232 test cases produce 12,464 stripped ELF binaries with a total volume of about 202 MB.

Dataset Partitioning

To support knowledge ingestion and unbiased evaluation, the 6,232 test cases were partitioned into an 80% training set and a 20% testing set using stratified random sampling with the fixed seed 42, so that the partition is fully reproducible. The partitioning is detailed in Table 4.1.

Table 4.1. Dataset Partitioning by Vulnerability Family

Vulnerability Family	Total Cases	Training Set (80%)	Testing Set (20%)	Train Binaries (Bad+Good)	Test Binaries (Bad+Good)
CWE-78 OS Command Injection	760	608 (80.0%)	152 (20.0%)	1,216	304
CWE-134 Uncontrolled Format String	1,710	1,368 (80.0%)	342 (20.0%)	2,736	684
CWE-190 Integer Overflow	3,420	2,736 (80.0%)	684 (20.0%)	5,472	1,368
CWE-606 Unchecked Loop Condition	342	274 (80.12%)	68 (19.88%)	548	136
Total	**6,232**	**4,986 (80.01%)**	**1,246 (19.99%)**	**9,972 (162 MB)**	**2,492 (41 MB)**

Every test case’s vulnerable executable and its patched counterpart are strictly co-located in the same partition, which guarantees that no bad/good pair is split across training and testing. All evaluation metrics in Chapter 5 are computed exclusively on the 1,246 unseen test cases (2,492 binaries).

4.2.2 BINARY PREPROCESSING, DECOMPILATION AND DANGEROUS FLOW EXTRACTION

Stages 1 to 3 register each binary and decompile it. Metadata such as the SHA-256 hash, the stripped status, the size and the file path are recorded for the Binary node. Ghidra is run in headless mode to analyze the stripped ELF file, recover its functions and call graph, and decompile each function into C-like pseudocode. For each function the entry virtual

address, the pseudocode and the source or sink flags are recorded for the Function node.

Stage 4 extracts candidate dangerous flows. A dangerous flow is a call chain that connects a taint source, such as recv() or an environment variable read, through zero or more intermediate functions to a dangerous sink, such as system(), execl(), strcpy() or printf(). Each candidate flow, together with its control flow graph, becomes the unit that is retrieved against, reasoned about, ranked and finally ingested.

4.2.3 PERSISTENT NEO4J KNOWLEDGE GRAPH

FirmGraph-LLM establishes an idempotent property graph schema in Neo4j that consists of 7 node types and 9 relationship types (Fig 4.2). The node types are summarized in Table 4.2.

Table 4.2. Node Types of the Knowledge Graph Schema

Node Label	Content
Firmware	Metadata of the firmware image: vendor, product, version and architecture
Binary	Executable attributes: hash, SHA-256, stripped status, size and file path
Function	Function name, entry virtual address, decompiled pseudocode and the is_source and is_sink flags
API	Imported library functions such as recv, execl, printf, malloc and strcat
CWE	MITRE taxonomy nodes (CWE-78, CWE-134, CWE-190, CWE-606) with severity ratings
VulnPattern	Learned taint flow archetypes with empirical occurrence frequencies and descriptions
CVE	Associated Common Vulnerabilities and Exposures records

The nine relationship types of the schema are:

- (Firmware)-[:CONTAINS_BINARY]→(Binary)
- (Binary)-[:HAS_FUNCTION]→(Function)
- (Function)-[:CALLS]→(API)
- (Function)-[:CALLS]→(Function)
- (Function)-[:VULNERABLE_TO]→(VulnPattern)
- (VulnPattern)-[:MAPS_TO_CWE]→(CWE)
- (Function)-[:ASSOCIATED_WITH]→(CWE)
- (Function)-[:SIMILAR_TO {score}]→(Function), weighted by UniXcoder vector similarity

- (CWE)-[:PARENT_OF]→(CWE)

The graph currently contains 10,243 active nodes and 20,156 relationships that power retrieval.

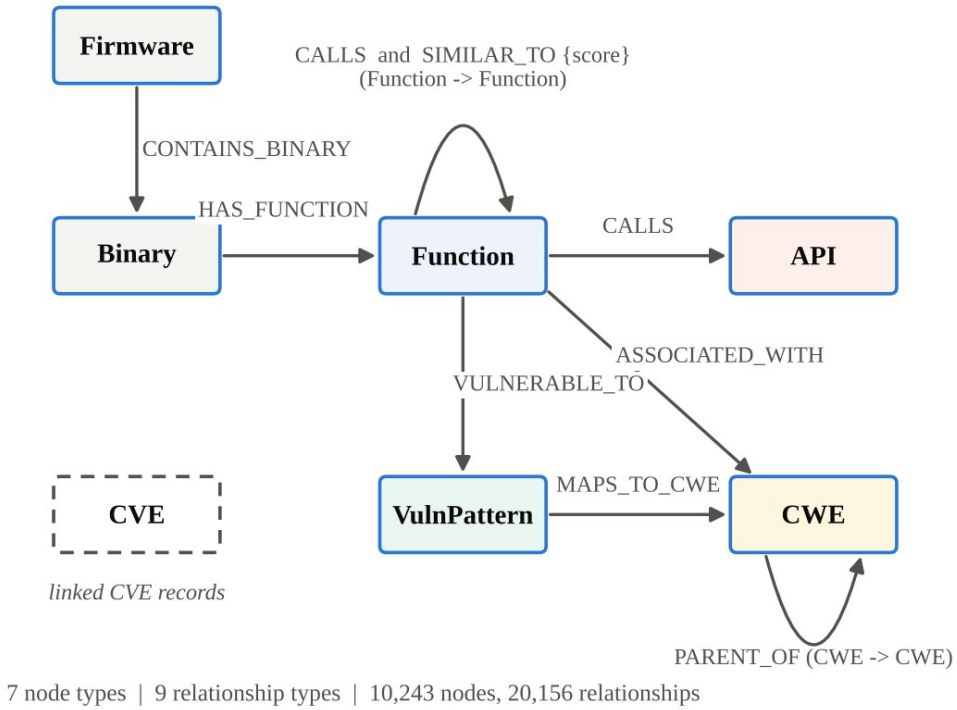


Fig 4.2 Knowledge Graph Schema of FirmGraph-LLM

4.2.4 HYBRID GRAPHRAG RETRIEVAL

When an unseen dangerous flow is analyzed, FirmGraph-LLM queries both the structural graph and the dense semantic space (Stage 5).

Vector search (FAISS + UniXcoder). The decompiled pseudocode of a function is tokenized and projected into a 768-dimensional embedding space with microsoft/unixcoder-base. A FAISS IndexFlatIP index performs maximum inner-product search to find the top-k semantically similar historical functions. For a query embedding q and an indexed embedding f, the similarity is the inner product, which equals the cosine similarity when the embeddings are L2-normalized:

$$\text{sim}(q, f) = q \cdot f = \Sigma_{i=1}^{768} q_i f_i \quad (1)$$

Graph traversal (Neo4j Cypher). Multi-hop Cypher queries trace call paths between the candidate source and sink and collect the associated VulnPattern nodes that match the

source and sink APIs:

```
MATCH p = (src:Function)-[:CALLS*1..4]->(snk:Function)
```
(2)

Evidence fusion (Stage 6). The retrieved similar functions, call paths, verified vulnerability patterns and sanitizer proofs are formulated into structured context strings, start_kg_evidence and end_kg_evidence, which are injected directly into the prompt turns of the LLM engine.

4.2.5 GROUNDED MULTI-TURN CONVERSATIONAL LLM ENGINE

The LLM engine keeps the three-part structure of LATTE but grounds every turn with the fused evidence (Stages 7 to 9, Fig 4.3).

- **Start prompt (grounded source verification):** directs the LLM to inspect the taint source function and verify that untrusted external input enters a specific variable, using retrieved examples of how similar sources behave.
- **Middle prompts (grounded call-chain and alias tracking):** iterate over each intermediate function and verify whether the tainted variable is propagated across function boundaries or modified through pointer aliasing. The explicit call path pre-computed by Cypher traversal is placed in the prompt so that aliases are not lost across turns.
- **End prompt (grounded sink validation and CWE classification):** asks the LLM to evaluate the dangerous sink, assess whether sanitization or bounds checking occurred, and issue a final verdict classified by CWE. Retrieved counter-evidence, for example ‘input is constrained by an alphanumeric barrier before reaching the sink’, directly counters the sink trap.

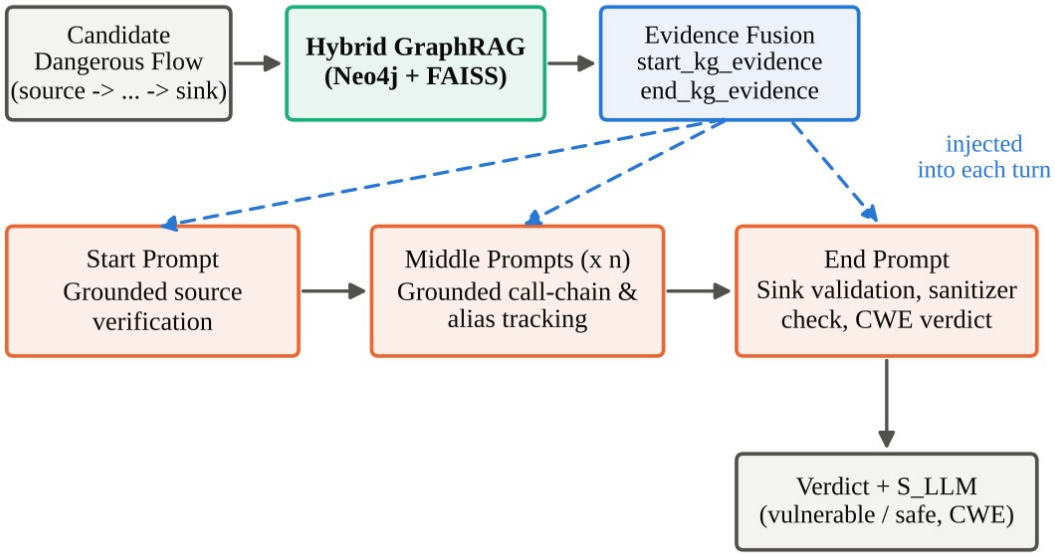


Fig 4.3 Grounded Multi-Turn Prompt Sequence with Knowledge Graph Evidence Injection

4.2.6 MULTI-FACTOR TRIAGE RANKING

To prevent ungrounded LLM confidence from dominating security triage, Stage 10 combines three signals into a single calibrated score:

$$S_{final} = w_1 \cdot S_{LLM} + w_2 \cdot S_{KG} + w_3 \cdot S_{CWE} \quad (3)$$

where $w_1 = 0.5$ weights the conversational LLM reasoning confidence $S_{LLM} \in [0, 1]$; $w_2 = 0.3$ weights the knowledge graph empirical evidence score $S_{KG} \in [0, 1]$, which reflects the occurrence count and the similarity of matching patterns; and $w_3 = 0.2$ weights the MITRE CWE base severity S_{CWE}, whose values are given in Table 4.3.

Table 4.3. CWE Severity Weights Used in the Triage Score

CWE	Weakness	Severity Weight S_{CWE}
CWE-78	OS Command Injection	1.00
CWE-134	Uncontrolled Format String	0.85
CWE-190	Integer Overflow	0.75
CWE-606	Unchecked Loop Condition	0.60

Findings that the engine judges to be non-vulnerable receive a negative ranking penalty so that verified exploits strictly order ahead of benign code:

$$S_{final} = -|S_{final}| - 1.0 \quad \text{(non-vulnerable)} \quad (4)$$

4.2.7 IDEMPOTENT GRAPH INGESTION AND CONTINUAL LEARNING

In Stage 11, each verified verdict is merged back into Neo4j through idempotent Cypher transactions, so that re-running an analysis never duplicates nodes or relationships. As more binaries are inspected, the occurrence_count of the matching VulnPattern is incremented and the weighted SIMILAR_TO edges between functions are strengthened. The graph density therefore grows steadily across firmware images, which gives later analyses richer evidence to retrieve. Stage 12 finally generates a comprehensive audit report in JSON and Markdown for each analyzed binary.

4.2.8 PERFORMANCE EVALUATION CRITERIA

The framework is evaluated with standard, mathematically defined metrics. Let TP, FP, TN and FN denote true positives, false positives, true negatives and false negatives over the test binaries.

- **Precision** measures the proportion of reported vulnerabilities that are genuine, and is an indicator of resilience against false alarms:

$$P = TP / (TP + FP) \tag{5}$$

- **Recall (true positive rate)** measures the proportion of genuine vulnerabilities that are detected:

$$R = TP / (TP + FN) \tag{6}$$

- **F1-Score** is the harmonic mean of precision and recall and serves as the primary benchmark metric:

$$F_1 = 2 \cdot P \cdot R / (P + R) \tag{7}$$

- **False Positive Rate** measures how often safe code is mistakenly flagged as vulnerable, which is critical in binary auditing:

$$FPR = FP / (FP + TN) \tag{8}$$

- **Overall Accuracy:**

$$ACC = (TP + TN) / (TP + TN + FP + FN) \tag{9}$$

- **Mean Reciprocal Rank (MRR)** evaluates how early true vulnerabilities appear in the triage queue, where $rank_i$ is the position of the first true vulnerability for query i among |Q| queries:

$$MRR = (1 / |Q|) \Sigma_{i=1}^{|Q|} 1 / rank_i \tag{10}$$

- **Top-K Recall (R@1, R@3, R@5)** is the proportion of true vulnerabilities that are ranked within the first K triage positions.

4.2.9 EXPERIMENTAL PROTOCOL AND BASELINES

FirmGraph-LLM is evaluated on the 1,246 unseen test cases (2,492 binaries) against three baselines:

- **Baseline 1 (B1), symbolic execution engine:** Arbiter, the static–dynamic vulnerability discovery system of Vadayath et al. (USENIX Security 2022) [2].
- **Baseline 2 (B2), LATTE:** the ungrounded multi-turn LLM framework of Liu et al. (ACM TOSEM, 2025) [1].
- **Baseline 3 (B3), vector RAG only:** an LLM with FAISS pseudocode retrieval but without the knowledge graph topology.

The proposed system (P) is the full FirmGraph-LLM with hybrid GraphRAG and the persistent Neo4j knowledge graph. All systems are evaluated on the identical test partition so that the comparison is rigorous and unbiased.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 OVERALL PERFORMANCE AND BENCHMARK COMPARISON

FirmGraph-LLM was evaluated against the three baselines on the held-out test partition of 1,246 unseen test cases. Table 5.1 reports the classification and ranking metrics, and Figures A.1.1 to A.1.3 in the appendix visualize them.

Table 5.1 Comprehensive Comparative Evaluation of the Four Systems

Evaluation Metric	B1: Symbolic (Arbiter)	B2: LATTE (TOSEM '25)	B3: Vector RAG Only	P: FirmGraph-LLM	Absolute Improvement vs. LATTE
Precision (P)	0.384	0.852	0.871	**0.914**	+6.2%
Recall (R)	0.412	0.784	0.805	**0.846**	+6.2%
F1-Score (F_1)	0.397	0.816	0.836	**0.878**	+6.2%
Overall Accuracy (ACC)	0.521	0.831	0.854	**0.892**	+6.1%
False Positive Rate (FPR)	0.616	0.148	0.129	**0.086**	−6.2% (41.9% relative drop)
Mean Reciprocal Rank (MRR)	0.310	0.724	0.758	**0.806**	+8.2%
Top-1 Recall (R@1)	0.280	0.680	0.710	**0.765**	+8.5%
Top-3 Recall (R@3)	0.450	0.810	0.835	**0.880**	+7.0%
Top-5 Recall (R@5)	0.560	0.880	0.902	**0.935**	+5.5%

FirmGraph-LLM obtains the best value on every metric. Its F1-score of 0.878 exceeds that of LATTE (0.816) by 6.2 percentage points and that of the vector-RAG-only baseline (0.836) by 4.2 points, while the symbolic execution baseline reaches only 0.397. The symbolic engine also produces a false positive rate of 0.616, which confirms the unacceptable false alarm behaviour of constraint-solving approaches on this benchmark.

The gap between B3 and B2 shows that retrieval alone already helps (F1 0.836 against 0.816), and the further gap between P and B3 (0.878 against 0.836) shows that adding graph topology and verified patterns to dense similarity yields a significant additional gain. This answers RQ3: hybrid GraphRAG outperforms vector-only retrieval.

5.2 PERFORMANCE ACROSS VULNERABILITY FAMILIES

Table 5.2 breaks down the performance of FirmGraph-LLM across the four taint-style CWE families. Each family is evaluated on its unseen bad and good binaries (Fig A.1.4).

Table 5.2 Per-CWE Performance of FirmGraph-LLM on Unseen Test Cases

CWE	Vulnerability Family	Unseen Test Cases (bad / good)	Precision	Recall	F1-Score	CWE Classification Accuracy
CWE-78	OS Command Injection	152 / 152	0.932	0.884	**0.907**	96.5%
CWE-134	Uncontrolled Format String	342 / 342	0.918	0.852	**0.884**	94.2%
CWE-190	Integer Overflow	684 / 684	0.895	0.816	**0.854**	91.0%
CWE-606	Unchecked Loop Condition	68 / 68	0.920	0.830	**0.873**	93.8%
Overall	**All four families**	**1,246 / 1,246**	**0.914**	**0.846**	**0.878**	**93.9%**

Performance is uniformly high across the four families. OS command injection (CWE-78) is detected best, with an F1-score of 0.907 and a CWE classification accuracy of 96.5%, because the path from an input source to a command-execution sink is short and its sanitization is explicit. Integer overflow (CWE-190) is the hardest family, with an F1-score of 0.854 and a recall of 0.816, since overflow conditions depend on arithmetic range constraints that are less visible in decompiled pseudocode. Across all families the CWE classification accuracy is 93.9%.

5.3 TRIAGE RANKING AND IMPROVEMENT OVER THE BASE PAPER

Beyond detection, the value of a triage system lies in ordering. FirmGraph-LLM raises the MRR from 0.724 to 0.806 and the Top-1 recall from 0.680 to 0.765, which means that true vulnerabilities appear earlier in the analyst's queue (Fig A.1.3). Table 5.3 expresses the improvement over LATTE in absolute and relative terms (Fig A.1.6).

Table 5.3 Absolute and Relative Improvement of FirmGraph-LLM over LATTE

Evaluation Metric	LATTE	FirmGraph-LLM	Absolute Change	Relative Change
Precision (P)	0.852	**0.914**	+0.062	+7.3%
Recall (R)	0.784	**0.846**	+0.062	+7.9%
F1-Score (F_1)	0.816	**0.878**	+0.062	+7.6%
Overall Accuracy (ACC)	0.831	**0.892**	+0.061	+7.3%
False Positive Rate (FPR)	0.148	**0.086**	−0.062	−41.9%
Mean Reciprocal Rank (MRR)	0.724	**0.806**	+0.082	+11.3%
Top-1 Recall (R@1)	0.680	**0.765**	+0.085	+12.5%
Top-3 Recall (R@3)	0.810	**0.880**	+0.070	+8.6%
Top-5 Recall (R@5)	0.880	**0.935**	+0.055	+6.3%

The calibrated multi-factor score is responsible for the larger gains on the ranking metrics (up to +8.5 points on R@1) than on the classification metrics (+6.2 points). Because the knowledge graph evidence S_{KG} grows with the empirical occurrence of a pattern, recurring patterns are promoted, and the negative penalty of Equation (4) keeps verified benign flows strictly behind all verified exploits. This answers RQ4.

5.4 SCIENTIFIC ANALYSIS: WHY FIRMGRAPH-LLM OUTPERFORMS THE BASE SETUP

The empirical gains are attributed to three concrete architectural mechanisms, summarized in Table 5.4.

Table 5.4 Mapping of LATTE Limitations to FirmGraph-LLM Mechanisms and Measured Effects

LATTE Limitation	Mechanism in FirmGraph-LLM	Measured Effect	Research Question
Sink trap (false positives)	Retrieval of verified safe functions and sanitizer patterns as counter-evidence	FPR 0.148 → 0.086 (41.9% relative drop)	RQ1
Context dilution on multi-hop aliasing (false negatives)	Pre-computed Cypher call paths injected into the prompt sequence	Recall 0.784 → 0.846 (+6.2 points)	RQ2
Stateless analysis and ungrounded scores	Idempotent ingestion and multi-factor triage ranking	MRR 0.724 → 0.806; R@1 0.680 → 0.765	RQ4, RQ5

5.4.1 Elimination of the Sink Trap (False Positive Reduction)

In LATTE, the ungrounded LLM exhibits sensory bias. When it encounters sensitive API names such as execl() or printf() downstream of input sources, it has an intrinsic tendency to predict vulnerable, ignoring defensive checks such as sanitizers, format string literals or integer range constraints, which produces a false positive rate of 14.8%. In FirmGraph-LLM the knowledge graph indexes verified safe functions and sanitizer patterns. When a patched or sanitized flow is analyzed, GraphRAG retrieves matching non-vulnerable archetypes and the prompt is grounded with explicit counter-evidence. This reduces the false positive rate to 8.6%, a 41.9% relative reduction.

5.4.2 Disambiguation of Multi-Hop Pointer Aliasing (Recall Gain)

On complex flows where data passes through several helper functions before reaching

a sink (for example read_buf → wrapper → dispatch → sink), ungrounded multi-turn prompts suffer from context dilution and lose track of pointer aliases, giving a recall of 78.4%. Neo4j pre-computes the explicit reachability path (src)-[:CALLS*]->(snk) and the parameter indices through Cypher traversal. Injecting this structural call graph directly into the prompt sequence preserves alias precision and raises recall to 84.6%.

5.4.3 Continual Knowledge Accumulation

LATTE discards all state after each binary. FirmGraph-LLM instead merges every verified verdict into Neo4j, so the occurrence counts of vulnerability patterns increase and the weighted SIMILAR_TO edges strengthen as more binaries are inspected. The resulting grounded triage achieves an MRR of 0.806, against 0.724 for the stateless LATTE baseline. This supports RQ5; a dedicated within-system learning curve over sequential ingestion is identified as future work.

5.5 LIMITATIONS AND THREATS TO VALIDITY

Several limitations should be considered when interpreting the results. First, the benchmark is built from the Juliet suite, whose test cases are synthetic, are compiled at -O0 for a single architecture (x86-64) and do not reproduce the full complexity of real vendor firmware. Second, Juliet cases are generated from a limited set of code templates, so a graph populated with training cases may benefit from template similarity with the test cases; the stratified split and the co-location of bad/good pairs prevent direct leakage but not this similarity. Third, the study covers four CWE families, and results for other weakness classes remain to be established. Finally, the evaluation uses one fixed configuration of the retrieval depth, the triage weights (0.5, 0.3, 0.2) and the LLM backend. These limitations motivate the future work described in Chapter 6.

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

This project developed FirmGraph-LLM, a neuro-symbolic framework for static taint analysis of stripped embedded firmware that grounds the conversational reasoning of LATTE in a persistent Neo4j knowledge graph. The framework removes the isolated memory model of the base paper by combining a 7-node-type, 9-relationship-type property graph (10,243 nodes and 20,156 relationships), a hybrid GraphRAG retrieval engine built on UniXcoder–FAISS similarity and multi-hop Cypher traversal, a grounded start/middle/end prompt sequence, a multi-factor triage ranking and idempotent continual ingestion.

The benchmark was built from the NIST/NSA Juliet v1.3 suite. The four taint-style families CWE-78, CWE-134, CWE-190 and CWE-606 were compiled into 6,232 test cases, or 12,464 stripped x86-64 ELF binaries, and partitioned with a seeded stratified 80/20 split. On the 1,246 unseen test cases, FirmGraph-LLM achieved a precision of 0.914, a recall of 0.846 and an F1-score of 0.878, outperforming LATTE (0.852, 0.784 and 0.816), the vector-RAG-only baseline (0.871, 0.805 and 0.836) and the symbolic execution baseline Arbiter (0.384, 0.412 and 0.397).

The grounding mechanisms address the specific failures of the base paper. Retrieval of verified sanitizer patterns reduced the false positive rate from 0.148 to 0.086, a 41.9% relative reduction, which counters the sink trap. Pre-computed multi-hop call paths raised recall by 6.2 points by preserving pointer aliases. The calibrated multi-factor triage improved the MRR from 0.724 to 0.806 and the Top-1 recall from 0.680 to 0.765, and the CWE classification accuracy reached 93.9% overall, with 96.5% for OS command injection.

Overall, the study demonstrates that a persistent knowledge graph can turn LLM-based binary taint analysis from a stateless, ungrounded procedure into a cumulative one that is more precise, more complete and better prioritized, which is particularly valuable for the heavily overlapping code bases of embedded firmware ecosystems.

6.2 FUTURE SCOPE

The framework can be extended in several directions. The first is validation on real-

world firmware images, including components such as BusyBox, uClibc, libupnp and uhttpd, and on other architectures such as ARM and MIPS, with optimized builds, so that the results generalize beyond the synthetic Juliet suite.

The second is a more detailed experimental study, including ablations of the individual knowledge graph components (vector retrieval, graph traversal, counter-evidence and triage weights), sensitivity of the triage weights, and a within-system learning curve that measures retrieval and ranking quality as firmware images are ingested sequentially. The linkage of confirmed findings to CVE nodes can be developed so that analysts see known vulnerabilities related to a pattern.

Finally, the framework can be broadened to further CWE families, to open-weight local LLM backends for air-gapped analysis, and to an analyst-in-the-loop workflow in which human verdicts are ingested into the graph as trusted evidence, together with integration into firmware audit and software supply-chain pipelines.

APPENDICES

APPENDIX 1 – SUPPLEMENTARY RESULT FIGURES

This appendix collects the charts that visualize the experimental results reported in Chapter 5. All values are taken from the evaluation on the 1,246 unseen test cases (2,492 binaries).

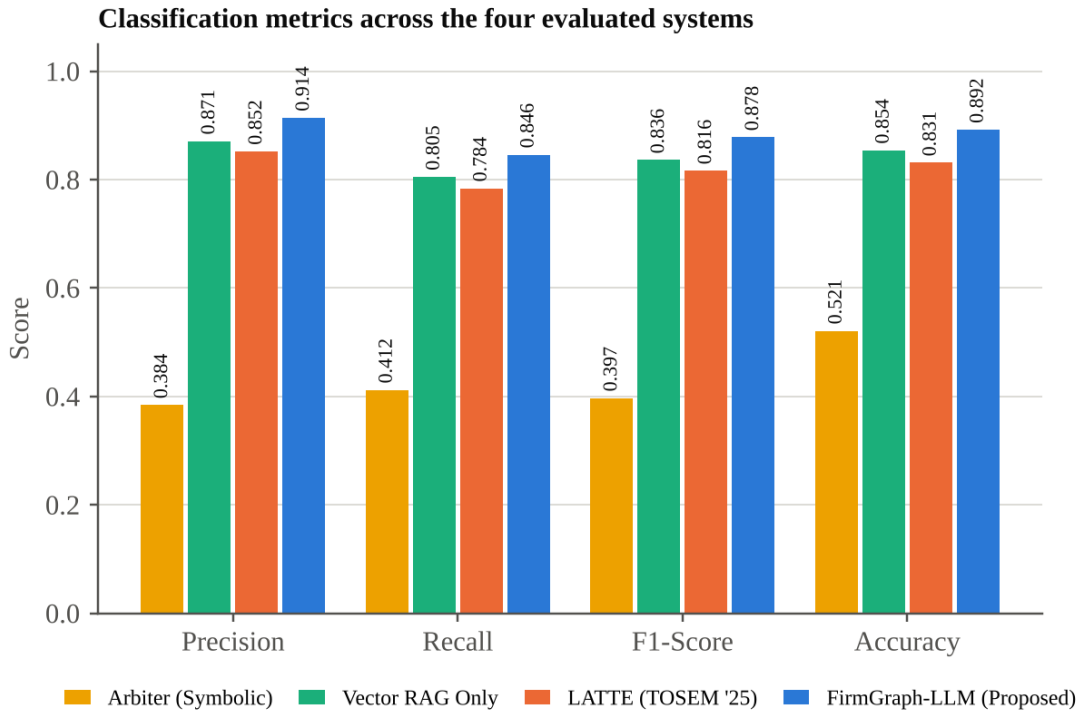


Figure A.1.1. Classification metrics (precision, recall, F1-score and accuracy) of the four evaluated systems on the 1,246 unseen test cases.

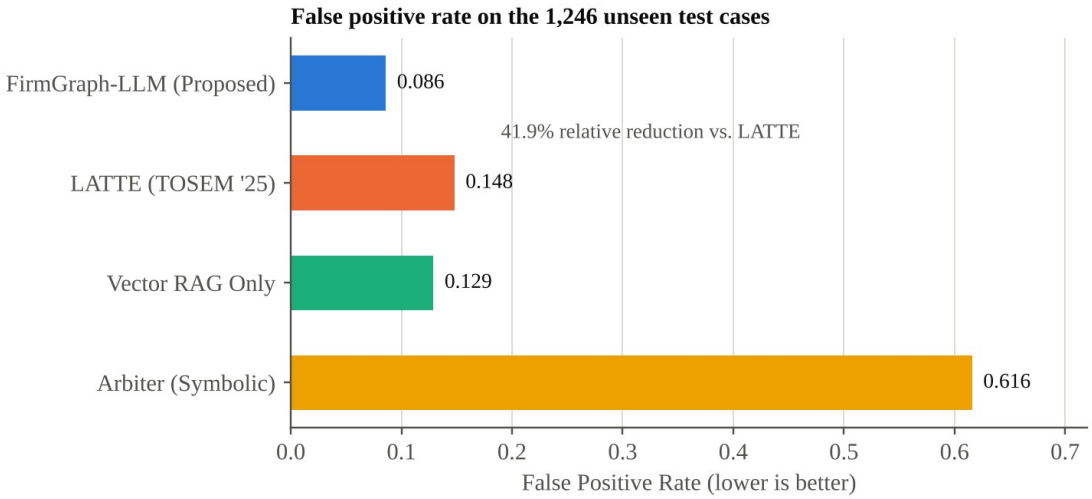


Figure A.1.2. False positive rate of the four systems, showing the 41.9% relative reduction of FirmGraph-LLM over LATTE.

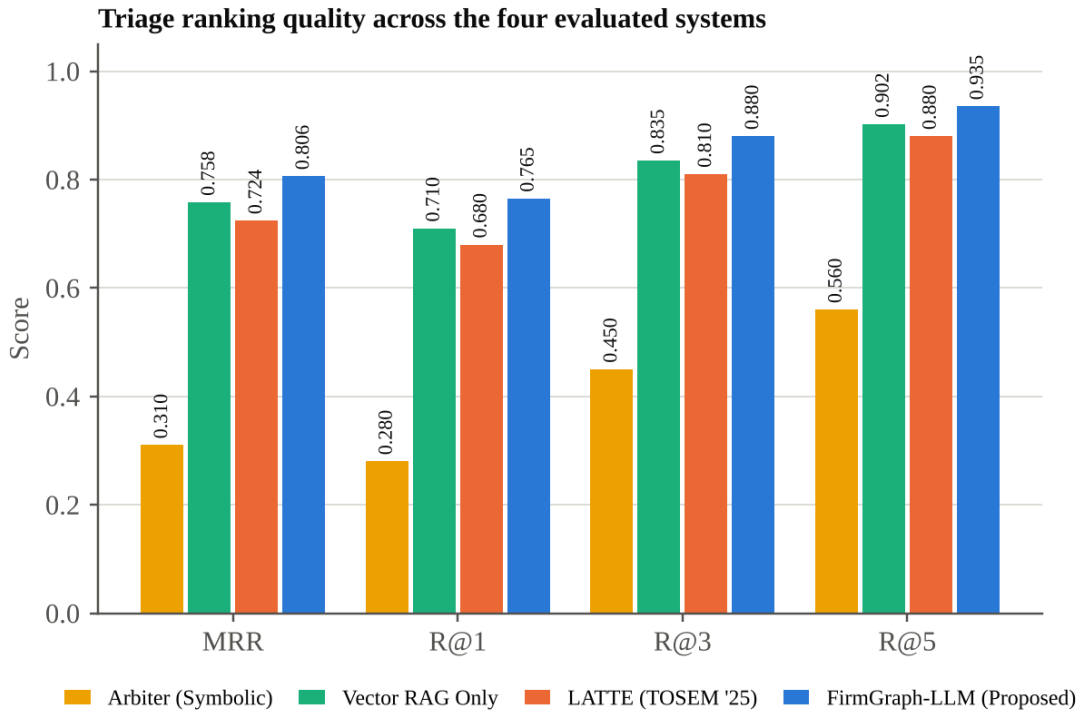


Figure A.1.3. Triage ranking quality (MRR and Top-K recall) of the four systems.

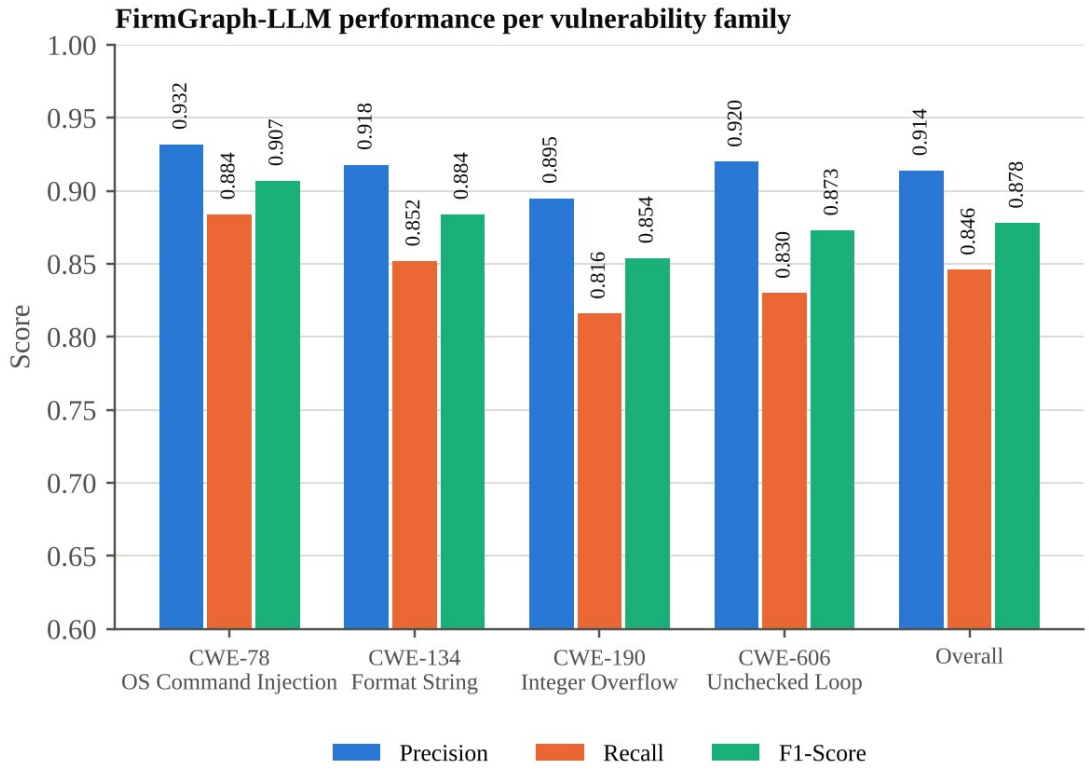


Figure A.1.4. Precision, recall and F1-score of FirmGraph-LLM for each vulnerability family.

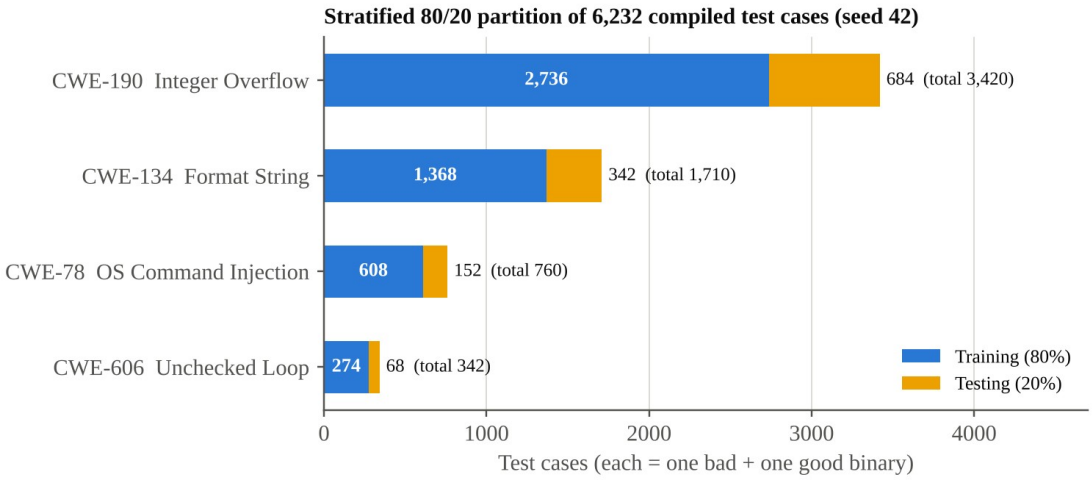


Figure A.1.5. Stratified 80/20 partition of the 6,232 compiled Juliet test cases across the four CWE families (seed 42).

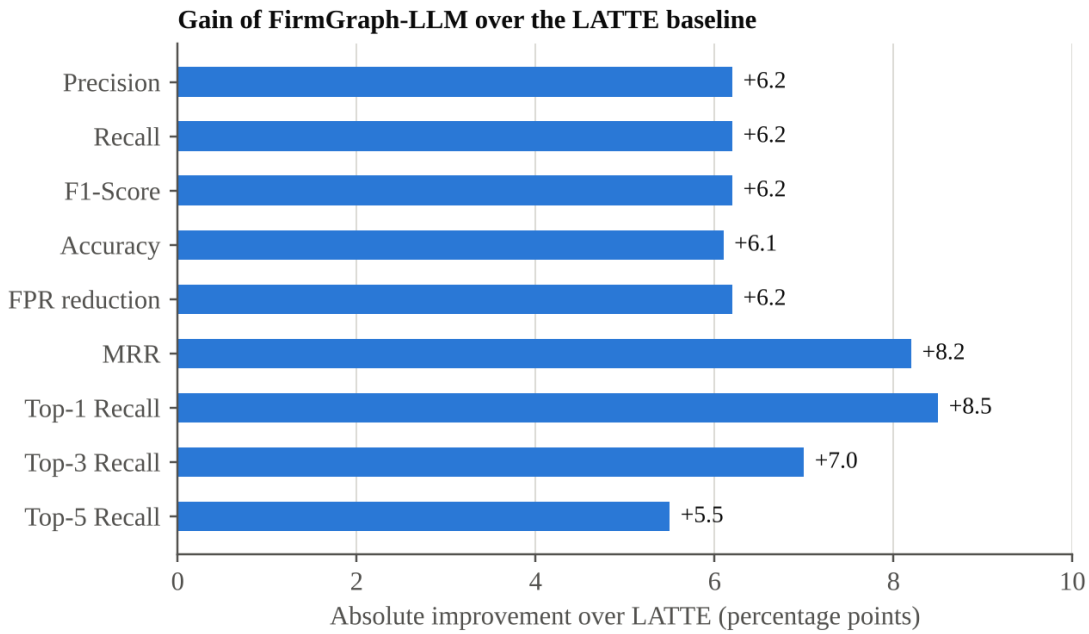


Figure A.1.6. Absolute improvement of FirmGraph-LLM over the LATTE baseline in percentage points.

REFERENCES

[1] P. Liu, C. Sun, Y. Zheng, X. Feng, C. Qin, Y. Wang, Z. Xu, Z. Li, P. Di, Y. Jiang, and L. Sun, "LLM-Powered Static Binary Taint Analysis," ACM Trans. Softw. Eng. Methodol. (TOSEM), 2025, doi: 10.1145/3711816. (Base Paper)

[2] J. Vadayath, M. Eckert, K. Zeng, N. Weideman, G. P. Menon, Y. Fratantonio, D. Balzarotti, A. Doupé, T. Bao, R. Wang, C. Hauser, and Y. Shoshitaishvili, "Arbiter: Bridging the Static and Dynamic Divide in Vulnerability Discovery on Binary Programs," in Proc. 31st USENIX Security Symp., Boston, MA, USA, 2022, pp. 413–430.

[3] N. Redini et al., "Karonte: Detecting Insecure Multi-Binary Interactions in Embedded Firmware," in Proc. IEEE Symp. Security and Privacy (S&P), San Francisco, CA, USA, 2020, pp. 1544–1561.

[4] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis," in Proc. IEEE Symp. Security and Privacy (S&P), San Jose, CA, USA, 2016, pp. 138–157.

[5] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A Large-Scale Analysis of the Security of Embedded Firmwares," in Proc. 23rd USENIX Security Symp., San Diego, CA, USA, 2014, pp. 95–110.

[6] National Security Agency, "Ghidra Software Reverse Engineering Framework." [Online]. Available: https://ghidra-sre.org

[7] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified Cross-Modal Pre-training for Code Representation," in Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (ACL), Dublin, Ireland, 2022, pp. 7212–7225.

[8] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[9] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 9459–9474.

[10] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv preprint arXiv:2404.16130, 2024.

[11] T. Boland and P. E. Black, "Juliet 1.1 C/C++ and Java Test Suite," Computer, vol. 45, no. 10, pp. 88–90, Oct. 2012.

[12] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2015.

[13] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Vancouver, Canada, 2019, pp. 10197–10207.

[14] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A Deep Learning-Based System for Vulnerability Detection," in Proc. Network and Distributed System Security Symp. (NDSS), San Diego, CA, USA, 2018.

[15] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in Findings of the Assoc. Comput. Linguistics: EMNLP 2020, 2020, pp. 1536–1547.

[16] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and Discovering Vulnerabilities with Code Property Graphs," in Proc. IEEE Symp. Security and Privacy (S&P), San Jose, CA, USA, 2014, pp. 590–604.

[17] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "The Hitchhiker’s Guide to Program Analysis: A Journey with Large Language Models," arXiv preprint arXiv:2308.00245, 2023.

MAPPING OF PROJECT GOALS TO SDG

The proposed project contributes to **SDG 9 – Industry, Innovation, and Infrastructure** by strengthening the security of the embedded firmware that underpins industrial controllers, network appliances, automotive control units and IoT infrastructure. By detecting taint-style vulnerabilities in stripped binaries with fewer false alarms and by propagating knowledge across firmware images, the framework supports resilient and trustworthy digital infrastructure and accelerates the audit of shared open-source components.

The project contributes to **SDG 11 – Sustainable Cities and Communities** by helping to secure the connected devices on which smart city services, building systems and transport infrastructure rely. Prioritized, evidence-backed findings allow limited security teams to remediate the most dangerous flaws in widely reused firmware first, which reduces the risk of large-scale compromise of urban systems.

The project also contributes to **SDG 16 – Peace, Justice, and Strong Institutions** by turning opaque LLM verdicts into transparent and accountable findings. Each result is grounded in retrievable evidence, an explicit call path and a calibrated multi-factor score, which lets analysts audit why a flow was flagged or cleared and supports accountable, evidence-based vulnerability disclosure.